Cluedo Game Project 2 Part 2 Documentation

Student: King Solijonov

Programming Language: Python

Project Type: Text-Based Logic and Deduction Game with Browser Frontend

Abstract

This project implements a digital version of Cluedo, also known as Clue, using Python and a browser-based frontend. The goal of the game is to allow the player to investigate a murder mystery by moving through rooms, making suggestions, observing refutations, recording evidence, and making a final accusation. Part 2 expands the original setup by adding card dealing, suggestion and refutation logic, a deduction notebook, computer-controlled players, and a win/loss accusation mechanism.

Introduction

Cluedo is a classic deduction-based board game where players attempt to determine the murderer, weapon, and location of a crime. This project was created to demonstrate object-oriented programming, logical reasoning, game state management, and basic artificial intelligence concepts. The implementation uses classes to represent players, the mansion, and deduction tracking. The program simulates the main mechanics of Cluedo in a terminal interface and also includes a browser-based frontend for easier demonstration.

Game Rules and Mechanics

At the start of the game, one character, one weapon, and one room are randomly selected as the hidden solution. These cards are removed from the deck. The remaining cards are dealt to the human player and computer players. Players move between connected rooms. When a player makes a suggestion, the other players are checked in turn order. If another player has a card matching the suggested character, weapon, or room, that player refutes the suggestion by showing a card. The human player can use this information to update their deductions. A player may make an accusation at any time. A correct accusation wins the game, while an incorrect accusation eliminates the player.

Key Features Implemented

- Mansion layout with connected rooms.
- Character definitions and starting positions.
- Weapon definitions.
- Random solution selection.
- Card deck creation and card dealing.
- Player movement between valid connected rooms.
- Suggestion handling with suspect, weapon, and room.
- Refutation handling when another player has a matching card.
- Deduction notebook for tracking seen cards, unknown cards, and suggestion history.
- Final accusation logic with win/loss outcomes.
- Browser-based frontend for visual gameplay and presentation demonstration.

Design Choices

The project uses object-oriented programming to keep the code organized and maintainable. The Player class stores each player's character, location, hand, and active status. The Mansion class stores room connections and validates movement. The DeductionNotebook class stores information learned by the human player. The CluedoGame class controls the overall game flow. This design separates responsibilities and makes future upgrades easier, such as adding a graphical interface or smarter AI deduction.

Suggestion and Refutation Logic

Suggestions are represented as a tuple containing a character, weapon, and room. When a suggestion is made, the program checks the other players in turn order. The first player who has a matching card refutes the suggestion. If the human player made the suggestion, the shown card is revealed and added to the deduction notebook. If a computer player makes a suggestion, the notebook still records that a refutation occurred, but hidden information is not unfairly exposed to the human player.

Deduction System

The deduction notebook tracks all cards the player has seen, cards that are still unknown, and a history of suggestions and refutations. This supports logical reasoning because the player can compare known cards against unknown cards and use refutation history to narrow down the possible solution. The notebook is accessible during the player's turn and is also shown in the frontend interface.

Frontend Interface

The frontend interface was added as an optional enhancement to improve usability and presentation quality. It includes a visual mansion board, highlighted current room, movement buttons, dropdowns for suggestions and accusations, a message box for game feedback, and a deduction notebook. This makes the project easier to demonstrate during a screen recording or final presentation.

Testing Scenarios

The game was tested using multiple gameplay scenarios. First, the game setup was tested to ensure that a hidden solution is selected and solution cards are not dealt to any player. Second, room movement was tested to ensure the player can only move to connected rooms. Third, suggestions were tested to verify that matching cards trigger refutations. Fourth, accusations were tested to confirm that correct accusations win the game and incorrect accusations end or eliminate the player. Finally, the deduction notebook was tested to confirm that seen cards, unknown cards, and suggestion history update correctly.

Challenges Faced

One challenge was designing refutation logic that follows Cluedo rules without revealing too much information. Another challenge was making the deduction notebook useful while still preserving hidden information from computer players. Managing turn order and active player status also required careful control flow. The final implementation addresses these issues through separate helper methods and clear state tracking.

Stability and Reliability

The program uses input validation for menu choices and number selections. The game state is stored in structured classes, which reduces the chance of inconsistent data. Since the Python version does not depend on external libraries, it is simple to run on most machines with Python installed. The frontend version uses plain HTML, CSS, and JavaScript, making it easy to open in a browser without additional installation. The game is stable for repeated playthroughs and supports the required Part 2 mechanics.

Future Improvements

Future improvements could include a more advanced graphical user interface, a backend API integration, smarter AI opponents, saved game states, full multiplayer support, and more advanced deduction algorithms. A deployed web version could also make the game easier to share and demonstrate.

How to Run

To run the Python terminal version, open a terminal, navigate to the project folder, and run the following command:

```
python3 cluedo_game_part2.py
```

To run the browser frontend, navigate to the frontend folder and open the index.html file in a browser. On macOS, this can be done with:

```
cd frontend
open index.html
```

Sample Terminal Commands

```
cd KingSolijonov_Project2_Part2_SourceCode
python3 cluedo_game_part2.py
```

Gameplay Screenshots

The screenshots below demonstrate the browser-based frontend, including the mansion board, current player status, movement options, suggestion/refutation feedback, deduction notebook, and accusation outcome.

Figure 1. Initial frontend view showing the mansion board, player cards, movement buttons, suggestion controls, and deduction notebook.

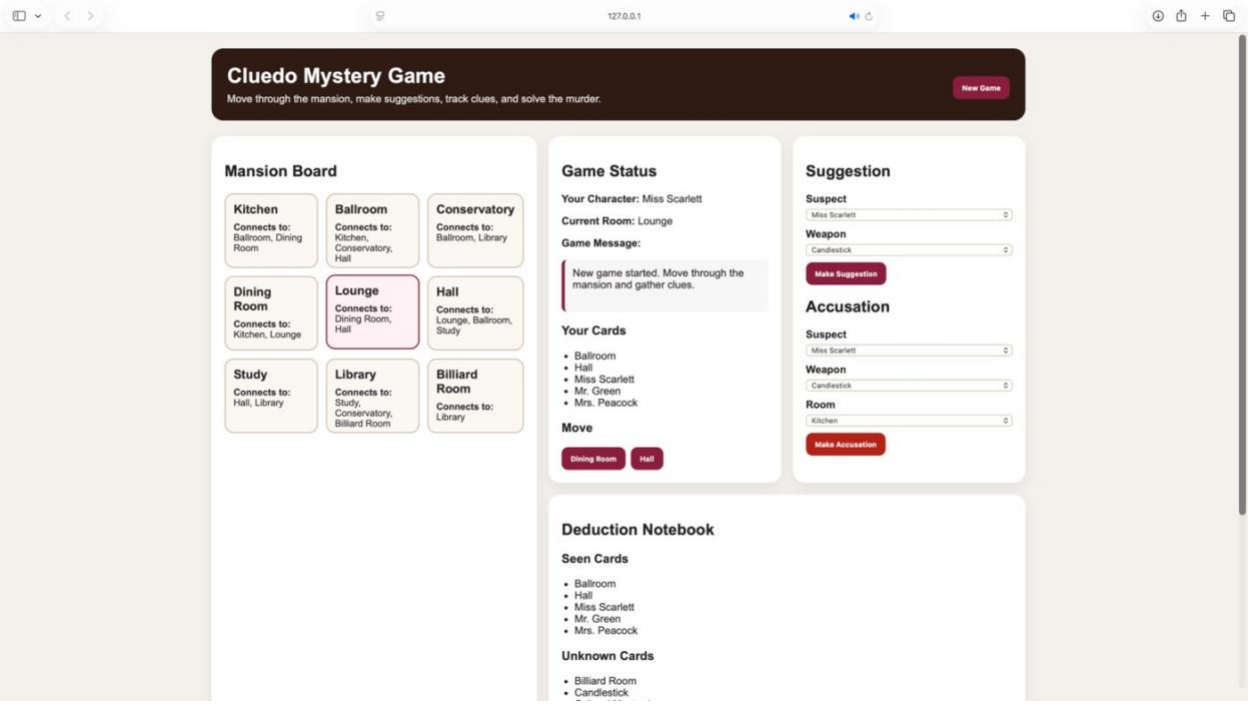

Figure 2. Deduction notebook view showing seen cards, unknown cards, and the suggestion/refutation history area.

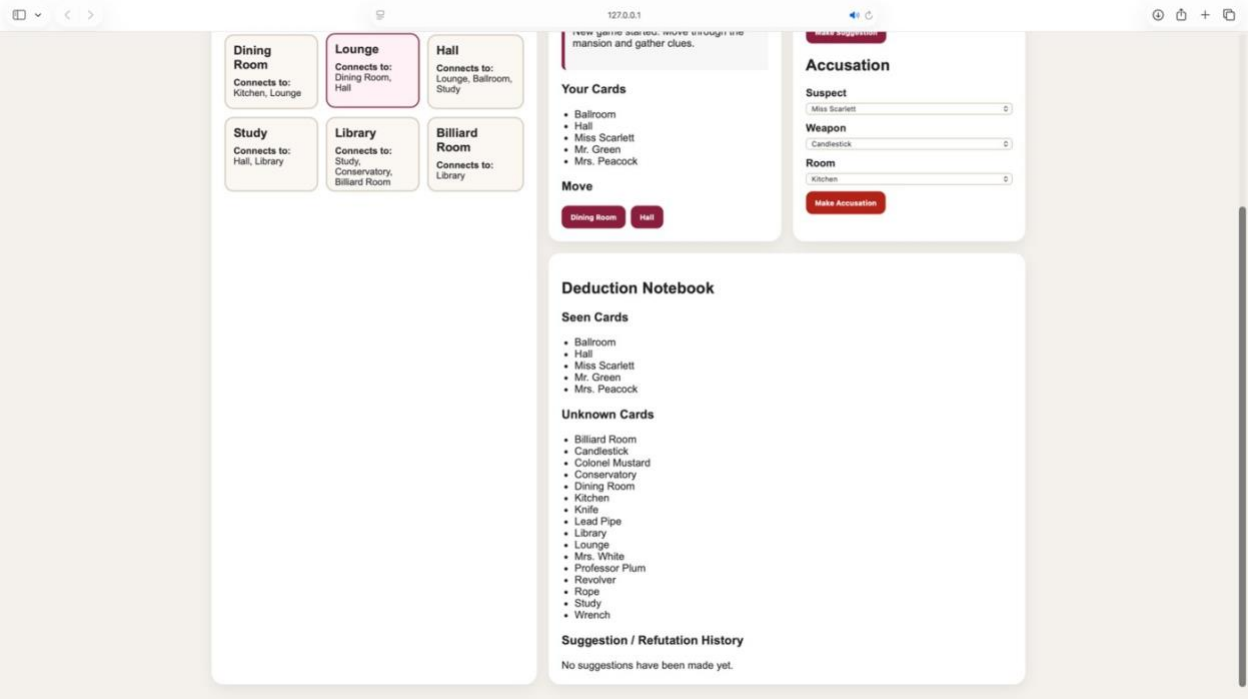

Figure 3. Example suggestion and refutation result after another player shows a matching card.

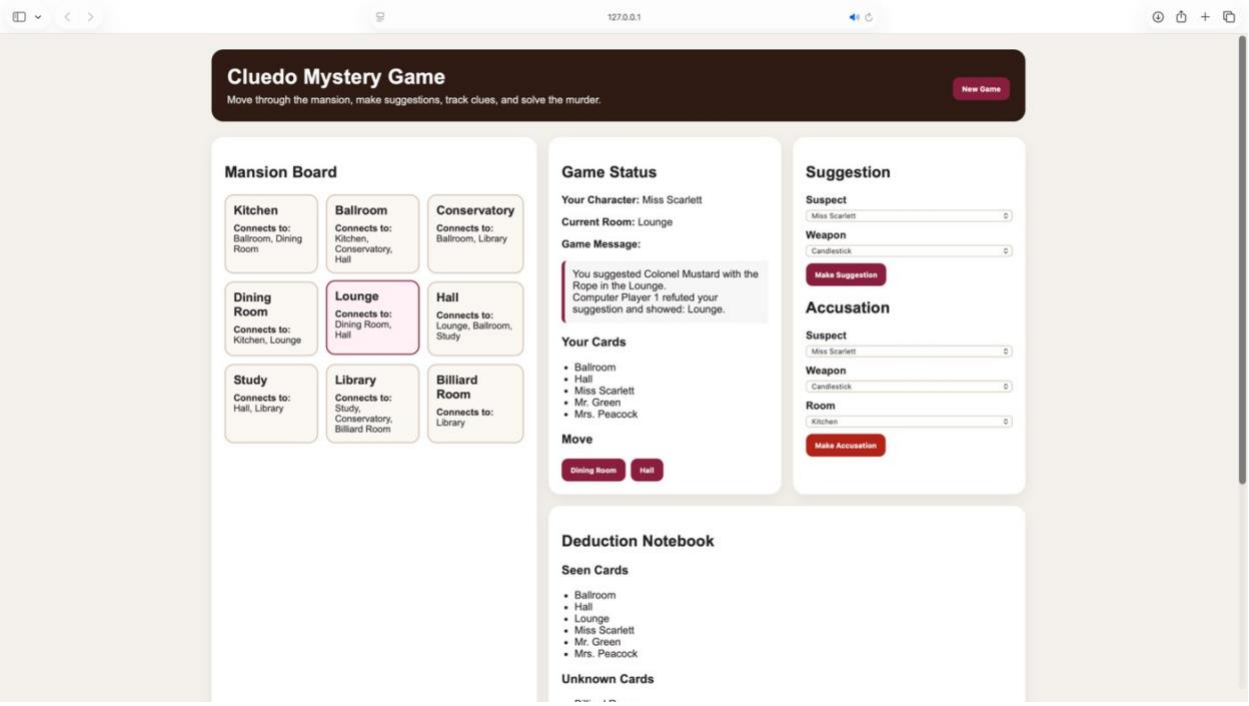

Figure 4. Example accusation result showing the game-over message and correct solution.